

Epidemic Spread Simulator

A global pandemic simulator where every parameter comes from real outbreak data — and the model is checked against what actually happened.

Susceptible Exposed Infected Recovered

The thing that turns this from a game into a graded data science project is grounding. Anyone can build a spreading-blob toy. What earns the marks is that your transmission rates are *fitted* from historical case curves, your travel network is a *real* flight route graph, your intervention effects are *regressed* from actual government policy data — and you can replay 2020 and show your simulation lands close to what really happened.

01 Data

Pick the datasets

Four core sources cover almost everything. All of them are free, well-documented, and citable in your report.

Dataset	What it gives you	Why you need it
OWID COVID-19 owid/covid-19-data	Daily cases, deaths, hospitalisations, testing and vaccinations for every country, one row per location per date	The spine of the whole project. It also bundles context variables — population, density, median age, GDP per capita, hospital beds per thousand, HDI — which become your per-country model inputs
OxCGRT (Oxford Government Response Tracker)	Daily stringency index plus individual flags: school closures, stay-at-home orders, travel bans	This is what lets you quantify intervention effects instead of guessing them. Partly bundled inside OWID already
OpenFlights openflights.org/data	67,663 routes between 3,321 airports on 548 airlines	Aggregate to country→country edge weights and you have a real global mobility network — the thing that moves your disease between countries
Natural Earth country polygons	GeoJSON boundaries + centroids	The map layer, and the geographic distances you'll compare against network distance

Worth adding if you have room

- WHO FluNet — weekly influenza surveillance. Gives you a second disease with completely different seasonality, which is strong evidence your model generalises rather than being tuned to COVID.
- Historical outbreak parameters — published R_0 and case-fatality figures for 1918 influenza, Ebola 2014, measles, SARS. These become your preset diseases in the app.

Two things to flag in your report

OWID's GitHub mirror stopped updating in August 2024 — use their data catalog for the current file, and pin whichever snapshot you use so results are reproducible. And OpenFlights route data is a 2014 snapshot. Neither is a problem for a modelling project, but examiners notice when you spot your own data's limitations before they do.

02 Pipeline

Make the Linux layer real, not decorative

Your submission form asks specifically how Linux and Python integrate. The honest answer is to actually build the ingestion in shell and keep the modelling in Python, so the boundary is genuine.

```
epidemic-sim/
├── scripts/
│   ├── fetch_data.sh      # curl the sources, log timestamps
│   ├── verify.sh         # sha256sum - skip reprocessing unchanged files
│   └── prefilter.awk      # strip columns before pandas touches 400MB
├── src/
│   ├── ingest/           # pandas cleaning, country code harmonisation
│   ├── analysis/         # EDA, Rt estimation, regressions
│   ├── models/           # SEIRD, metapopulation, Markov mutation
│   └── api/              # FastAPI + websocket
├── notebooks/           # the analysis story
├── web/                 # frontend
├── Makefile             # make data | make fit | make serve
└── Dockerfile           # python:3.11-slim (Debian base)
```

```
└── requirements.txt
```

Concretely, the Linux side does: `curl/wget` retrieval with retry and logging; `sha256sum` change detection so you don't rebuild everything on every run; `awk/cut` pre-filtering of large CSVs before Python loads them; a `Makefile` tying the stages together; `cron` for a weekly data refresh; `venv` for dependency isolation; Docker on a Debian base image; and Git driven from the CLI. That's a real ETL story you can walk an examiner through.

03 Analysis

Find the trends — this is where the marks are

The brief asks for trends and patterns, so treat this as a first-class deliverable, not homework before the fun part. Seven things worth hunting, roughly in order of how impressive they are to present:

Wave structure

Run peak detection (`scipy.signal.find_peaks`) over 7-day-smoothed case curves per country. How many waves did each country have, how far apart, and did waves propagate geographically or hit everywhere at once?

Early growth rates

Log-linear fit on the exponential phase of each country's first wave gives you doubling times. The spread of that distribution across countries is itself a finding.

Effective reproduction number over time

Estimate R_t per country (Cori method or a simpler Wallinga–Teunis approach). This is the single most useful output of the whole analysis phase — it becomes the β your simulator uses.

Intervention effect size

Regress change in R_t against the OxCGRT stringency index with a lag of roughly 7–14 days. You end up able to say something like "a 10-point rise in stringency corresponded to an X% drop in R_t " — and then your lockdown slider in the app uses that number instead of a made-up one.

Why fatality varies by country

Correlate case fatality ratio against median age, hospital beds per thousand, and HDI. This explains why the same disease produces very different death tolls, and it justifies making your simulator's mortality country-specific.

Spread follows the flight network, not the map

Take each country's date of first reported case and plot it against (a) geographic distance from the origin and (b) shortest-path distance on the flight graph. The network distance should correlate far better. It's a genuinely striking result, it's easy to visualise, and it's the empirical justification for your whole metapopulation design.

Epidemic archetypes

Extract shape features from each country's curve — peak timing, peak height, number of waves, area under curve — and cluster with K-Means or hierarchical clustering. You'll get recognisable groups: early suppressors, double-wave countries, sustained-transmission countries. Nice map to show.

04 Modelling

Build the engine in four layers

Build these in order. Each one works on its own, so if you run out of time you stop at whatever layer you've reached and still have something demonstrable.

Layer 1 — SEIRD core, per country

Compartments $S \rightarrow E \rightarrow I \rightarrow R$, with D branching off I . Solved as ODEs with `scipy.integrate.solve_ivp`. Parameters: β (transmission rate), σ (1/incubation period), γ (1/infectious period), μ (fatality). $R_0 = \beta/\gamma$. Add a V compartment for vaccination once the basic version is validated against a single country.

Layer 2 — Metapopulation coupling

Every country runs its own SEIRD. A mobility matrix built from the flight route data moves a fraction of each compartment between countries at every timestep. This is the actual architecture real pandemic forecasting models use, so you can cite that approach in your report. Travel restrictions become a simple scaling of the mobility matrix.

Layer 3 — Markov chains

This is your differentiator, and it connects straight to your probability and stochastic processes coursework. Use it twice, for two genuinely different jobs:

- Individual state transitions. A discrete-time Markov chain with transition matrix P over $\{S, E, I, R, D, V\}$. R and D are absorbing states, so you can do absorption analysis and derive expected epidemic duration analytically — a nice theoretical result to sit alongside the simulation.
- Mutation and variants. A second chain whose states are variants. Transition probability scales with the current infected population, so more infections genuinely means more chances to mutate. Each variant carries its own $(R_0, \text{fatality, immune escape})$ vector, and when a transition fires the active parameters swap mid-simulation. This gives

you the mutation mechanic you wanted, and it's mathematically defensible rather than a game gimmick.

Layer 4 — Stochastic seeding

ODEs break at low numbers — you end up simulating 0.004 of an infected person. So below a threshold of roughly 10–20 cases, run a branching process with Poisson draws; above it, hand over to the deterministic solver. This is standard practice in the field, it makes early spread appropriately unpredictable, and it means two runs of the same scenario won't be identical.

Why this ordering matters

Layer 1 alone is a working single-country simulator. Layers 1+2 give you the global map. Layer 3 adds the coursework tie-in and the mutation feature. Layer 4 is polish. Nothing later is a prerequisite for anything earlier, so there's no point where a delay leaves you with nothing to submit.

What the simulator actually looks like

Your rough idea — build a disease, drop it anywhere, watch it spread, tune parameters live — is the right shape. Here's a fuller version of it, built as seven screens.

Disease lab

Start with presets fitted from your own analysis: COVID-19 (Wuhan 2020), Omicron, 1918 influenza, seasonal flu, Ebola, measles. Each loads real R_0 , incubation, and fatality values, so the user's first experience is anchored in something true.

Then the custom builder: sliders for R_0 (0.5–18), incubation period, infectious period, case fatality ratio, asymptomatic fraction, mutation rate, and transmission mode — respiratory, contact, vector, or waterborne, which changes how population density and climate modify β . A live readout computes the expected final attack rate from the analytical final-size equation, so you get instant feedback before you ever press play. A small marker shows where your invented disease sits relative to the real ones in your dataset.

Patient zero

Click any country on the globe to seed it. Set initial case count and whether it starts urban or rural. The panel shows that country's real stats pulled from OWID — population, density, median age, hospital beds per thousand — and makes clear that these feed the model. Seeding in a dense, well-connected hub behaves visibly differently from seeding somewhere isolated, and that difference is data-driven rather than scripted.

Simulation view

The main screen. An animated choropleth world map where colour encodes prevalence, with toggleable overlays for deaths, vaccination coverage, and active restrictions. Flight arcs light up when the disease crosses a border, drawn from the actual OpenFlights routes — this is both the best-looking thing in the app and a direct visualisation of your model's mechanics.

Around it: time controls (play, pause, 1× to max speed, day counter and date), a global stats bar (total infected, active cases, deaths, countries affected, dominant variant, global R_t), live SEIRD curves, and a hospital capacity-versus-demand chart.

Event log

A running feed: first case detected in a new country, a variant emerging with its trait changes, a government closing borders, a health system exceeding capacity. This is a small feature that does a lot of work — it makes the simulation legible instead of just pretty, and it gives you natural talking points during your demo.

Interventions

Applied globally or per country: lockdown (scales β), border closure (scales the mobility matrix), mask mandates, test-and-isolate (accelerates $I \rightarrow R$), vaccination rollout with a start day and daily rate, and hospital surge capacity.

Two details worth building. First, effectiveness should decay over time to model compliance fatigue — and you can calibrate that decay from the OxCGRt data rather than inventing it. Second, let interventions be *scheduled* on a trigger ("lock down when national active cases exceed 50,000"), because that turns the app into something you can run experiments with.

Variant tree

A small branching diagram showing each variant, where and when it emerged, and how its traits differ from its parent. Immune-escape variants push a fraction of the recovered compartment back into susceptible, which produces the reinfection waves people will recognise from real life.

Counterfactual mode

Run the same disease twice with different intervention timing and overlay the curves. "What if the lockdown had started two weeks earlier?" is the single most valuable thing this app can do, because it's an analytical use rather than entertainment — and it's exactly the kind of question your regression from phase 3 gives you the right to answer.

Validation view

Build this even though it isn't fun. Replay actual 2020–21 COVID with fitted parameters and the real OxCGRt policy timeline, overlay simulated against reported cases per country, and report RMSE or MAPE. It is the difference

between a simulator and a toy, and it's the screen that will convince an examiner the rest is real.

Optional, if you want a game loop

A challenge mode where the user plays public health authority against a scenario and is scored on deaths plus economic cost. Fun, and it makes the demo memorable — but only build it once validation works.

06Stack

What to build it with

Two viable paths. Pick based on how much frontend time you want to spend.

The good version

FastAPI backend with the simulation engine as a pure NumPy module, streaming state over WebSockets so the map updates tick by tick. Frontend in

React with deck.gl or MapLibre for the globe, plus a charting library for the curves. More work, but it's the one that looks like a product rather than a class assignment — and given you already do UI design, it plays to your strengths.

The fast fallback

Streamlit with a Plotly choropleth and a time slider. You can have something working in a weekend. It won't feel real-time, but it proves the model. Honestly the best move is to build this in week 1 as a prototype regardless, then upgrade.

Everything else

pandas, NumPy, SciPy and statsmodels for analysis; scikit-learn for the clustering; NetworkX for the flight graph; GeoPandas for spatial work; Parquet or SQLite for processed data; Docker for deployment on any free Linux host.

On performance

Roughly 200 countries $\times$ 6 compartments $\times$ one daily timestep is nothing. Vectorised in NumPy, a two-year global simulation runs in milliseconds. So "real time" here means you're deliberately throttling playback so it's watchable — not waiting on compute. Worth knowing early, because it means you can afford stochastic runs and Monte Carlo ensembles later without redesigning anything.